

COMPLETE GUIDE: Deploying a Web Application Using Docker (Frontend + Backend)

OVERALL FLOW (Big Picture Diagram)

```
graph TD; A[Your Laptop] --> B[Create GitHub Repos]; B --> C[Push Code (Frontend + Backend)]; C --> D[Login to Cloud Server (Putty)]; D --> E[Git Clone Repo]; E --> F[Create .env file]; F --> G[Docker Build]; G --> H[Docker Compose Up]; H --> I[Application Running on IP:PORT];
```

Your Laptop
↓
Create GitHub Repos
↓
Push Code (Frontend + Backend)
↓
Login to Cloud Server (Putty)
↓
Git Clone Repo
↓
Create .env file
↓
Docker Build
↓
Docker Compose Up
↓
Application Running on IP:PORT

PART 1 — BEFORE DEPLOYMENT (On Your Local System)

1 Create Two GitHub Repositories

You need:

```
your-project-backend  
your-project-frontend
```

Push your working code into both.

Make sure:

- No errors
- `.env` file NOT pushed to GitHub
- `package.json` exists
- Project runs locally

EXPECTED PROJECT STRUCTURE

Backend Structure (Example)

```
backend/  
|  
├─ src/  
├─ package.json  
├─ package-lock.json  
├─ Dockerfile  
├─ docker-compose.yml  
├─ .dockerignore  
└─ .env (not pushed)
```

Frontend Structure (Example React/Vite)

```
frontend/  
|  
├─ src/
```

```
|— public/
|— package.json
|— package-lock.json
|— Dockerfile
|— docker-compose.yml
|— .dockerignore
|— .env (not pushed)
```

PART 2 — REQUIRED DOCKER FILES

BACKEND DOCKER FILES

1 Backend Dockerfile

Create file:

Dockerfile

Paste:

```
# Use official Node image
FROM node:20-alpine

# Create app directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy project files
```

```
COPY . .

# Expose backend port
EXPOSE4000

# Start backend
CMD ["npm","start"]
```

2 Backend .dockerignore

Create:

```
.dockerignore
```

Paste:

```
node_modules
npm-debug.log
.git
.env
```

3 Backend docker-compose.yml

Create:

```
docker-compose.yml
```

Paste:

```
version: '3.8'

services:
  backend:
    build: .
```

```
container_name: backend_container
ports:
  - "4000:4000"
env_file:
  - .env
restart: always
```

FRONTEND DOCKER FILES

1 Frontend Dockerfile (React / Vite Production Build)

Create:

Dockerfile

Paste:

```
# Build Stage
FROM node:20-alpine as build

WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production Stage
FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
```

```
CMD ["nginx","-g","daemon off;"]
```

2 Frontend .dockerignore

```
node_modules
.git
.env
dist
```

3 Frontend docker-compose.yml

```
version: '3.8'

services:
  frontend:
    build: .
    container_name: frontend_container
    ports:
      - "3000:80"
    restart: always
```

PART 3 — SERVER DEPLOYMENT (Using Putty)

BACKEND DEPLOYMENT STEPS

Step 1 — Login to Server

Open Putty and connect:

```
ssh username@your_server_ip
```

Step 2 — Create Directory

```
mkdir backend-app  
cd backend-app
```

Step 3 — Clone Backend Repo

```
git clone https://github.com/yourusername/backend-repo.git
```

Step 4 — Enter Folder

```
cd backend-repo
```

Step 5 — Create .env File

```
nano .env
```

Add:

```
PORT=4000  
DATABASE_URL=your_database_url  
API_KEY=your_api_key  
SUPABASE_URL=your_url  
SUPABASE_ANON_KEY=your_key
```

Press:

```
CTRL + X  
Y
```

```
Enter
```

Step 6 — Build Docker

```
docker compose build
```

Step 7 — Start Backend

```
docker compose up -d
```

Now backend runs at:

```
http://your_server_ip:4000
```

Backend deployment done 

FRONTEND DEPLOYMENT STEPS

Step 1 — Create Folder

```
mkdir frontend-app  
cd frontend-app
```

Step 2 — Clone Frontend Repo

```
git clone https://github.com/yourusername/frontend-repo.git
```

Step 3 — Enter Folder

```
cd frontend-repo
```

Step 4 — Create .env

```
nano .env
```

Add:

```
VITE_BACKEND_URL=http://your_server_ip:4000
```

Save and exit.

Step 5 — Build

```
docker compose build
```

Step 6 — Start

```
docker compose up -d
```

Frontend now runs at:

```
http://your_server_ip:3000
```

Frontend deployment done 

CHECK IF EVERYTHING IS RUNNING

Check containers:

```
docker ps
```

You should see:

```
backend_container  
frontend_container
```

USEFUL COMMANDS

Stop containers:

```
docker compose down
```

Restart:

```
docker compose restart
```

View logs:

```
docker logs backend_container
```

WHAT A BEGINNER MUST UNDERSTAND

Concept	Meaning
Docker	Packs app + dependencies
Dockerfile	Recipe to build container
docker-compose	Runs container
.env	Secret keys
Port mapping	Server:Container
git clone	Downloads code
build	Prepares image
up -d	Runs in background

COMPLETE DEPLOYMENT FLOW DIAGRAM

Create GitHub Repo



Push Working Code



Add Dockerfile



Add docker-compose.yml



Push to GitHub



Login to Server



mkdir project



git clone



nano .env



docker compose build



docker compose up -d



Access IP:PORT

FINAL RESULT

Backend → <http://IP:4000>

Frontend → <http://IP:3000>

Your app is LIVE 🚀

Prompts to Create Docker files in Antigravity

prompt 1 :

now go through the entire codebase and explain me everything and all the major files that are required. , Give it to me in a diagrammatic style

prompt 2 :

Now , I am going to deploy this on docker in my putty ,

SO create me all the necessary files required

docker , docker-compose , .dockerignore and ensure everything is thoroughly checked and created

SO when I start from the git clone on my docker and compose and build

It should run smoothly and also add port number which doesn't use in my existing ports

note : put the port numbers in putty which were already used

use command

docker ps

prompt 3 :

Just go through the package.json and ensure we covered all the dependencies

required and won't run into any other problems while doing docker compose build